

Explicação Código

Contexto

O sistema de Votação ao todo têm cerca de 15 funções, funções com funcionalidades de apenas exibir o conteúdo na tela, funções de menu para fazer o controle das opções do menu, entre outras funções abaixo:



Função para Exibir

- `exibir_votos_pendentes()`
- `exibir_candidatos()`
- `exibir_apuracao()`
- `exibir_menu_principal()`

Função de Menu

- `menu_mesario()`
- `menu_eleitor()`
- `menu_administrador()`

Função do Sistema

- `cadastrar_novo_cpf()`
- `votar()`
- `apurar_votos()`
- `autorizar_voto()`

Função Auxiliares

- validar_senha_mesario()
- validar_senha_admin()
- limpar_tela()

Função Principal

- sistema_votacao()

Pre-requisito

Estude isso antes de continuar:

- Dicionário em Python
- For em Python
- If em Python
-

Função Sistema Votação

Contexto: Essa é a função principal do sistema, ela que vai controlar o que o sistema faz.

Cabeçalho da Função Principal

```
264     # Função do Sistema Principal
      1 usage
265     def sistema_votacao():
266         while True:
267             limpar_tela()
268             exibir_menu_principal()
269             opcao = int(input("Escolha uma opção: "))
270
```

- Linha 265: define o nome da função para sistema_votacao()
- Linha 266: abre um laço de repetição como sempre verdadeiro.

📄 Informação

Gerando um loop infinito o sistema sempre vai permanecer aberto até que seja interrompido pela comando `break`.

- Linha 267: Chama a função para limpar a tela.
- Linha 268: Chama a função para exibir o menu de opções para o usuário.

```
def exibir_menu_principal():  
    print("🌟🌸🌸 ----- ELEIÇÃO 2024 ----- 🌸🌟🌟")  
    print("1 🗳️ Mesário")  
    print("2 🗳️ Eleitor")  
    print("3 🗳️ Administrador")  
    print("4 🗳️ Sair")  
    print("🌟🌸🌸-----🌸🌟🌟")
```

Essa função apenas mostra na tela as opções. 1- Mesário, 2- Eleitor, 3- Administrador e 4- Sair.

linha 269: O sistema cria uma *variável*: *opcao* e chama a função `input("Escolha uma opção")`, essa função é responsável por capturar e retornar para a *variável* *opcao* o valor que é digitado pelo usuário, no caso se ele digitou 1, 2, 3...

Corpo da Função Principal

Agora que o usuário digitou algum número, a gente tem que fazer a condição.

Caso opção 1:

Digite "1" para executar as opções de mesário.

```

if opcao == 1:
    limpar_tela()
    if eleicao_encerrada:
        print("X Eleição está encerrada, se deseja a
    else:
        menu_mesario()
    input("Aperte qualquer tecla para continuar...")

```

Em resumo, `limpar_tela()` limpa a tela apagando o menu anterior, se(if) `eleicao_encerrada`(For verdadeira) ele não deixa acessar o menu do mesário, mostrando a mensagem que a eleição está encerrada. se não(else) o sistema chama `menu_mesario()`.

Caso opção 2:

Digite "2" para abrir a tela de votação.

```

elif opcao == 2:
    limpar_tela()
    if eleicao_encerrada:
        print("X Eleição está encerrada, se deseja a
    else:
        menu_eleitor()
    input("Aperte qualquer tecla para continuar...")

```

Opção dois é semelhante a opção 1, a diferença que caso a `eleicao_encerrada` for falso ele chama a o `menu_eleitor()` ao invés do `menu_mesario()`

Caso opção 3:

Digite "3" para abrir a tela do administrador.

```

elif opcao == 3:
    limpar_tela()
    menu_administrador()

```

essa opção apenas chama o `menu_administrador()`, até porque ela pode ser acessada independente da eleição está aberta ou fechada.

Caso opção 4:

Digite "4" para encerrar o sistema.

```
elif opcao == 4:
    print("Encerrando o sistema.")
    break
```

Essa é a opção mais importante do sistema, lembra que no cabeçalho da função principal usamos um `while True` para executar infinitamente? então, quando o usuário quer sair do sistema ele escolhe essa opção para encerrar o WHILE com o break;

Função Mesário

O `menu_mesario()` precisa ser acessado com senha, então a linha 180 ela pede para o usuário digitar(`input`) uma senha e guarda na `_variável` senha.

```
179 def menu_mesario():
180     senha = input("🔑 Informe a senha do mesário: ")
181     if validar_senha_mesario(senha):
182         print("👤 Mesário autenticado.")
183         while True:
184             limpar_tela() # Limpa a tela para uma interface mais limpa
185             print("🌸🌸🌸 ----- TELA DO MESÁRIO ----- 🌸🌸🌸")
186             print("1 🔑 Autorizar voto")
187             print("2 🚪 Sair")
188             print("🌸🌸🌸----- 🌸🌸🌸")
189
190             opcao = int(input("Escolha uma opção: "))
```

- linha 181: chama a função `validar_senha_mesario` e passa o valor digitado pelo usuário, função validar senha, verifica se o usuário digitou a senha correta:

```
def validar_senha_mesario(senha):  
    return senha == "mesario123"
```

Nesse caso ela verifica se o usuário digitou "mesario123", se deseja mudar a senha basta mudar "mesario123" para qualquer senha.

- linha 182: Aparece a mensagem Mesário autenticado. Caso o usuário digitou a senha certa!



- linha 183: Mais um while True, porque é outro menu, o menu do mesário.

Informação

Quando criamos um sistema de menus encadeados, como o "Menu Principal" que leva a submenus como "Menu Mesário" ou "Menu Administrador", a estrutura comum é usar loops (`while`) para permitir que o usuário escolha opções repetidamente. A lógica básica é:

- **Menu Principal:** Um loop `while` principal que exibe as opções e permite ao usuário escolher o submenu ou funcionalidade que deseja acessar.
- **Submenus:** Cada submenu também terá seu próprio loop `while` que mantém o submenu aberto até que o usuário escolha sair dele (geralmente com uma opção de "voltar").

Aqui temos apenas duas opções ou sair desse menu e volta para o `menu_principal` ou o usuário escolhe a opção 1 - `Autorizar Votos`

Função Autorizar Votos

```

def autorizar_voto():
    exibir_votos_pendentes()
    cpf = input("Digite o CPF do eleitor: ")
    # Pergunta se deseja autorizar o voto
    for eleitor in eleitores:
        if eleitor["cpf"] == cpf:
            print(f"\n🔍 Validação do voto para o CPF: {cpf} 🔍")
            resposta = input("Autorizar Voto? [1- Sim | 2- Não]\n")

            if resposta == "1":
                eleitor["validado"] = True
                eleitor["valido"] = True
                print("✅ Voto autorizado com sucesso! 🎉")
            elif resposta == "2":
                eleitor["validado"] = True
                eleitor["valido"] = False
                print("❌ Voto não autorizado. ❌")
            else:
                print("🚫 Resposta inválida. Por favor, escolha 1 ou 2.")

    input("Pressione qualquer tecla para continuar...")

```

Primeiro a função chama a função `exibir_votos_pendentes()` que exibe caso haja, os votos que estão pendentes.

```

def exibir_votos_pendentes():
    print("🌟:----- Votos Pendentes para Validação -----🌟")

    # Exibe os eleitores que precisam de validação
    votos_pendentes = False
    for eleitor in eleitores:
        if eleitor["votou"] and not eleitor["validado"]:
            print(f"👤 Eleitor - CPF: {eleitor['cpf']}")
            votos_pendentes = True

    # Verifica se há votos pendentes para validação
    if not votos_pendentes:
        print("🙅 Nenhum voto pendente para validação! 🙅")
        input("Pressione qualquer tecla para continuar...")
    else:
        print("🌟:----- 🌟")

```

Temos uma variável `flag votos_pendentes` que começa com o valor temporário com `False`.

`FOR` percorre todo os eleitores

```
{"cpf": cpf, "votou": False, "validado": False, "valido": False, "candidato": -1}
```

Aqui tem o conceito de `dicionário` em `Python`, é bom pesquisar sobre qualquer dúvida.

Mas basicamente o `for` verifica se `"votou"` é `True` e `"validado"` é `false`, o que significa que o eleitor votou, porém o seu voto ainda não foi validado.

Informação

Quando o vetor estiver percorrendo e encontrar pelo menos um eleitor para validar o voto, então nossa `flag votos_pendentes` vai ser `True`, então não vai ser mostrado a mensagem em baixo de *Nenhum voto pendente...*

VOLTANDO para Autorização do Voto, agora que sabemos que tem votos pendentes ou não para validar, a função pergunta para o mesário qual `cpf`(dos que apareceram) ele quer validar o voto.

```
cpf = input("Digite o CPF do eleitor: ")
```

E agora temos mais um `for` para percorrer novamente os eleitores, e encontrar o `CPF` escolhido:


```

# Pergunta se deseja autorizar o voto
for eleitor in eleitores:
    if eleitor["cpf"] == cpf:
        print(f"\n🔍 Validação do voto para o CPF: {cpf} 🔍")
        resposta = input("Autorizar Voto? [1- Sim | 2- Não]\n")

        if resposta == "1":
            eleitor["validado"] = True
            eleitor["valido"] = True
            print("✅ Voto autorizado com sucesso! 🎉")
        elif resposta == "2":
            eleitor["validado"] = True
            eleitor["valido"] = False
            print("❌ Voto não autorizado. ❌")
        else:
            print("🚫 Resposta inválida. Por favor, escolha 1 ou 2.")

```

Imagine que ao vetor tenha 2 eleitores:

Eleitor 1



Dados:

cpf: 112,

voto_valido: ?,

validado: Não,

candidato_votou: SECRETO

Eleitora 2:



Dados:

cpf: 312,

votou: ?,

validado: Não,

candidato_votou: SECRETO

O que basicamente está acontecendo é CPF do eleitor 112 foi digitado? Sim?
Sim! O mesário confirmou o voto dele!

Então mude as variáveis para:

voto_valido: Sim,

validado: Sim.

```
if resposta == "1":  
    eleitor["validado"] = True  
    eleitor["valido"] = True  
    print("✓ Voto autorizado com sucesso! 🎉")
```

A mas a eleitora teve seu voto invalidado para o mesário, então mude para:

voto_valido: Não,

validado: Sim.

```
elif resposta == "2":  
    eleitor["validado"] = True  
    eleitor["valido"] = False  
    print("✗ Voto não autorizado. ✗")
```

Em resumo, a função menu_mesario, faz a validação dos votos se são aprovados ou reprovados.

Função Menu Eleitor

```
def menu_eleitor():  
    print("🌟🌸 ----- ÁREA DE VOTAÇÃO ----- 🌸🌟")  
    cpf = input("% Digite o CPF do eleitor (somente números): ").strip()  
  
    # Tenta cadastrar o novo CPF e, se bem-sucedido, permite votar  
    if cadastrar_novo_cpf(cpf):  
        exibir_candidatos()  
        numero_candidato = int(input("Digite o número do candidato: "))  
  
        resultado_voto = votar(cpf, numero_candidato)  
        print(f"\n🗳️ {resultado_voto} 🗳️")  
    else:  
        print("❌ Esse CPF já tem um voto cadastrado. ❌")
```

Nessa função o sistema pede o cpf do eleitor, e guarda no sistema com a função `cadastrar_novo_cpf()`

Logo depois chama a função `exibir_candidatos()`

E pede para o eleitor digita o número do candidato que ele deseja votar!

por fim, chama a função `votar(cpf, numero_candidato)` função responsável por salvar o voto do eleitor.

Função Votar

```
def votar(cpf, numero_candidato):
    # Verifica se o eleitor está na lista
    for eleitor in eleitores:
        if eleitor["cpf"] == cpf:
            if eleitor["votou"]:
                return "Voto não autorizado!"

            # Verifica se o candidato existe
            for candidato in candidatos:
                if candidato["numero"] == numero_candidato:
                    eleitor["votou"] = True
                    eleitor["candidato"] = numero_candidato
                    return "Voto registrado com sucesso!"

            return "Candidato não encontrado."

    return "Eleitor não encontrado."
```

A função `votar(cpf, numero_candidato)` simula o processo de votação de um eleitor em um sistema eleitoral. Ela faz o seguinte:

1. **Verifica o CPF do eleitor:** A função percorre uma lista chamada `eleitores` e verifica se o CPF (identificação do eleitor) passado como argumento corresponde ao de algum eleitor na lista.
2. **Verifica se o eleitor já votou:** Se encontrar o eleitor pelo CPF, ela verifica se esse eleitor já votou. Caso tenha votado, a função retorna a mensagem **"Voto não autorizado!"** e encerra.
3. **Verifica se o candidato existe:** Se o eleitor ainda não votou, a função verifica se o número do candidato informado (passado como `numero_candidato`) está na lista de `candidatos`.
4. **Registra o voto:** Se o candidato for encontrado, o voto é registrado. O status do eleitor muda para "já votou" e o número do candidato votado é salvo. A função retorna **"Voto registrado com sucesso!"**.
5. **Candidato não encontrado:** Se o candidato com o número fornecido não existir, a função retorna **"Candidato não encontrado."**

Resumindo, essa função cuida de registrar um voto, verificando se o eleitor já votou e se o candidato é válido.

Função Apurar Voto

```
def apurar_votos():
    contagem = {}
    for candidato in candidatos:
        numero = candidato["numero"]
        contagem[numero] = 0

    for eleitor in eleitores:
        if eleitor["valido"]:
            contagem[eleitor["candidato"]] += 1

    max_votos = max(contagem.values())

    vencedores = []
    for candidato in candidatos:
        numero = candidato["numero"]
        if contagem[numero] == max_votos:
            vencedores.append(candidato["nome"])

    if len(vencedores) > 1:
        vencedor_nome = "Empate"
    else:
        vencedor_nome = vencedores[0]
```

```
mensagens = []
total_votos = sum(contagem.values())

for candidato in candidatos:
    numero = candidato["numero"]
    nome = candidato["nome"]
    votos = contagem[numero]
    mensagens.append(f"Candidato {nome} recebeu {votos} votos.")

return mensagens, vencedor_nome, total_votos
```

A função `apurar_votos()` faz a contagem de votos de uma eleição e determina o vencedor. Aqui está uma explicação passo a passo de como ela funciona:

1. Inicializa a contagem de votos:

- Cria um dicionário chamado `contagem` onde cada chave é o número do candidato, e o valor é o número de votos que esse candidato recebeu.
- Um laço `for` percorre a lista de `candidatos` e define o número de votos de cada um como 0 no início.

2. Conta os votos válidos:

- A função percorre a lista de `eleitores`. Para cada eleitor, se o voto for considerado válido (indicador `valido`), o voto é contado para o candidato correspondente. O número do candidato é usado para atualizar a contagem de votos.

3. Encontra o máximo de votos:

- A função usa `max(contagem.values())` para descobrir quantos votos o candidato mais votado recebeu.

4. Determina o(s) vencedor(es):

- A função percorre novamente a lista de candidatos. Para cada candidato que recebeu o número máximo de votos, seu nome é adicionado a uma lista chamada `vencedores`.
- Se houver mais de um nome nessa lista (ou seja, empate), o vencedor é definido como **"Empate"**. Caso contrário, o nome do único vencedor é retornado.

5. Gera mensagens de votação:

- A função cria uma lista chamada `mensagens`, onde cada entrada é uma mensagem informando quantos votos cada candidato recebeu.
- Também calcula o total de votos válidos somando todos os votos registrados na contagem.

6. Retorna os resultados:

- A função retorna três coisas:
 - A lista de mensagens que informa a quantidade de votos recebidos por cada candidato.
 - O nome do vencedor (ou **"Empate"**, se houver empate).
 - O número total de votos válidos.

Essa função organiza a contagem dos votos, identifica o vencedor e gera um relatório dos resultados da eleição.